

LABORATORY REPORT

On

24MCADMP206 – ADVANCED DBMS LAB

SECOND SEMESTER

DEPARTMENT OF COMPUTER APPLICATIONS

(NBA Accredited)



NEHRU COLLEGE OF ENGINEERING AND RESEARCH CENTRE

AN AUTONOMOUS INSTITUTION AFFILIATED TO APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY, KERALA

(NAAC Re-Accredited with “A” Grade)

An ISO Certified Institution,

Pampady, Thiruvilwamala, Thrissur-680588

May 2026



NEHRU COLLEGE OF ENGINEERING AND RESEARCH CENTRE

AN AUTONOMOUS INSTITUTION AFFILIATED TO APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY, KERALA

(NBA Accredited, NAAC Re-Accredited with “A” Grade)

An ISO Certified Institution,

Pampady, Thiruvilwamala, Thrissur-680588



CERTIFICATE

**Certified that this is the bonafide Laboratory report of the subject
24MCADMP206 Advanced DBMS Lab in Second Semester KTU-MCA**

Practical done by.....

Register No:.....

during the academic year May 2026

Lab in-charge

Head of the Department

Internal Examiner

External Examiner

CONTENTS

S.NO	NAME OF THE PROGRAM	DATE	SIGN OF EXAMINER
1	DDL COMMANDS		
2	DML COMMANDS		
3	DCL COMMANDS		
4	USE OF SELECT, WHERE, GROUP BY & ORDER BY CLAUSE		
5	USE OF AGGREGATE FUNCTIONS		
6	JOIN OPERATIONS		
7	SET OPERATIONS		
8	PL/SQL PROCEDURE		
9	PL/SQL TRIGGER		
10	PL/SQL CURSOR		
11	PL/SQL FUNCTIONS		
12	DESIGN A MongoDB DATABASE		
13	CRUD OPERATIONS IN MongoDB		
14	AGGREGATE FUNCTIONS IN MongoDB		
15	REGULAR EXPRESSION IN MongoDB		
16	USER CREATION & ROLES IN MongoDB		
17	MongoDB ADMINISTRATION SECURITY & BACKUP		
18	FRONT END & NoSQL CONNECTIVITY		

Experiment No : 1

Aim :Write MySQL query that shows the execution of various DDL commands

Solution :

DDL commands:

- CREATE – is used to create the database or its objects (like table, index, function, views, store procedure and triggers).
- DROP – is used to delete objects from the database.
- ALTER-is used to alter the structure of the database.

Creation of a database :

```
CREATE DATABASE stu;
```

To use the database named as ‘stu’ from the list of database we need to execute the following command :

```
USE stu;
```

Creation of a table ‘students’ in a database named as ‘stu’ :

```
CREATE TABLE students (  
StudentIDint,  
Name varchar(255),  
Address varchar(255),  
City varchar(255)  
);
```

Alter Table - ADD Column named as Email to the students table:

```
ALTER TABLE students  
ADD Email varchar(255);
```

Alter Table - DROP Column named as City from the students table:

```
ALTER TABLE students  
DROP COLUMN City;
```

Alter Table –ALTER/MODIFY data type of Column named as StudentID from the students table:

```
ALTER TABLE students  
MODIFY COLUMN StudentIDvarchar(10);
```

Experiment No : 2

Aim : Write MySQL query that shows the execution of various DML commands

Solution :

DML commands:

- INSERT – is used to insert data into a table.
- UPDATE – is used to update existing data within a table.
- DELETE – is used to delete records from a database table.

INSERT INTO Statement:

```
INSERT INTO students (StudentID, Name, Address, Email)
```

```
VALUES ('a01', 'Ashish L', 'Palakkad', 'daksh@gmail.com');
```

```
INSERT INTO students (StudentID, Name, Address, Email)
```

```
VALUES ('a02', 'Vimal', 'Palakkad', 'vimu@gmail.com');
```

UPDATE Statement:

```
UPDATE students SET Name = 'Hari' WHERE StudentID= 'a01';
```

DELETE Statement :

```
DELETE FROM students;
```

Experiment No: 3

Aim: Write MySQL query that shows the execution of various DCL commands

Solution:

DCL commands:

- Grant - This command used to provide access or privileges on the database objects to the users
- Revoke – This command withdraw user privileges on database objects if any granted

GRANT Privilege :

```
CREATE USER ashish@localhost IDENTIFIED BY 'ashish123';
```

```
SHOW GRANTS FOR ashish@localhost;
```

```
GRANT SELECT ON TST TO ashish@localhost;
```

```
GRANT SELECT, INSERT, DELETE, UPDATE ON PST TO ashish@localhost;
```

```
GRANT ALL ON students TO ashish@localhost;
```

```
SHOW GRANTS FOR ashish@localhost;
```

Revoke Privilege :

```
REVOKE SELECT ON TST FROM ashish@localhost;
```

```
REVOKE SELECT, INSERT, DELETE, UPDATE ON PST FROM ashish@localhost;
```

```
REVOKE ALL ON students FROM ashish@localhost;
```

Experiment No: 4

Aim: Accessing database using SELECT, Filtering using WHERE, arranging records using GROUP BY etc.

Solution:

Creating a table :

```
CREATE TABLE EMP (  
EMP_NO int NOT NULL PRIMARY KEY,  
NAME varchar(255) NOT NULL UNIQUE,  
AGE int,  
EMP_ID int  
);
```

Inserting records in to the table EMP :

```
INSERT INTO EMP (EMP_NO, NAME, AGE, EMP_ID) VALUES (1, 'Ashish', 31, 27);  
INSERT INTO EMP (EMP_NO, NAME, AGE, EMP_ID) VALUES (2, 'Bibin', 35, 24);  
INSERT INTO EMP (EMP_NO, NAME, AGE, EMP_ID) VALUES (3, 'Sanjay', 45, 21);
```

SELECT Statement :

```
SELECT * FROM EMP;  
  
SELECT * FROM EMP WHERE NAME LIKE 'Ashish';
```

WHEREClause :

```
SELECT * FROM EMP WHERE AGE > 33;
```

GROUP BYClause:

```
SELECT `AGE` FROM `EMP` GROUP BY `AGE`;
```

HAVING Clause :

```
SELECT * FROM `EMP` GROUP BY `EMP_ID` HAVING AGE > 21 ;
```

ORDER BYClause:

```
SELECT * FROM EMP ORDER BY EMP_ID;
```

Experiment No: 5

Aim :Write MySQL query that shows the usage of aggregate functions

COUNT Function :

```
SELECT COUNT(NAME) FROM EMP;
```

SUM Function :

```
SELECT SUM(AGE) FROM EMP;
```

AVG function :

```
SELECT AVG(AGE) FROM EMP;
```

MAX Function :

```
SELECT MAX(AGE) FROM EMP;
```

MIN Function :

```
SELECT MIN(AGE) FROM EMP;
```


Experiment No: 6

Aim: Write a MySQL query that shows the working of various JOIN Operations

Solution:

JOINS

- MySQL INNER JOIN (or sometimes called simple join)
- MySQL LEFT OUTER JOIN (or sometimes called LEFT JOIN)
- MySQL RIGHT OUTER JOIN (or sometimes called RIGHT JOIN)

```
CREATE TABLE FACULTY (
```

```
FID int,
```

```
FNAME varchar(150),
```

```
FSUBJECT varchar(255)
```

```
);
```

```
INSERT INTO FACULTY (FID,FNAME,FSUBJECT) VALUES (1, 'Ashish', 'Design and  
Analysis of Algorithms');
```

```
INSERT INTO FACULTY (FID,FNAME,FSUBJECT) VALUES (2, 'Vimal', 'Basics of  
Electronics');
```

```
INSERT INTO FACULTY (FID,FNAME,FSUBJECT) VALUES (3, 'Daksh', 'Mobile  
Communication');
```

```
CREATE TABLE STUFAC (
```

```
SID int,
```

```
SNAME varchar(150),
```

```
SSEM varchar(255)
```

```
);
```

```
INSERT INTO STUFAC (SID,SNAME,SSEM) VALUES (1, 'Ajay', 'S3');
```

```
INSERT INTO STUFAC (SID,SNAME,SSEM) VALUES (2, 'Sanjay', 'S2');
```

```
INSERT INTO STUFAC (SID,SNAME,SSEM) VALUES (3, 'Biju', 'S2');
```

```
INSERT INTO STUFAC (SID,SNAME,SSEM) VALUES (4, 'Reju', 'S3');
```

```
SELECT FACULTY.FNAME, FACULTY.FSUBJECT, STUFAC.SSEM
```

```
FROM FACULTY
```

```
INNER JOIN STUFAC
```

```
ON FACULTY.FID = STUFAC.SID;
```

```
SELECT FACULTY.FNAME, FACULTY.FSUBJECT, STUFAC.SSEM
```

```
FROM FACULTY
```

```
LEFT JOIN STUFAC
```

```
ON FACULTY.FID = STUFAC.SID;
```

```
SELECT FACULTY.FNAME, FACULTY.FSUBJECT, STUFAC.SSEM
```

```
FROM FACULTY
```

```
RIGHT JOIN STUFAC
```

```
ON FACULTY.FID = STUFAC.SID;
```

Experiment No: 7

Aim: Write a MySQL query that shows the working of various SET Operations

Solution:

```
CREATE TABLE PST (  
  
ID int,  
  
Name varchar(255)  
  
);
```

```
CREATE TABLE TST (  
  
ID int,  
  
Name varchar(255)  
  
);
```

INTERSECT

```
SELECT DISTINCT ID FROM PST WHERE ID IN (SELECT ID FROM TST);
```

MINUS

```
SELECT ID FROM PST  
  
LEFT JOIN TST USING (ID)  
  
WHERE TST.ID IS NULL;
```

UNION ALL

```
SELECT * FROM PST UNION ALL SELECT * FROM TST;
```

UNION

```
SELECT * FROM PST  
  
UNION  
  
SELECT * FROM TST;
```

Experiment No: 8

Aim: Develop a PL/SQL Program that explains the Procedure concept

Solution:

Procedure:

```
create table Book(isbnvarchar(20), title varchar(20),author varchar(20),category  
varchar(20),price varchar(20),pidvarchar(20)) ;
```

```
insert into Book values ("100001","C Programming","Ajay","Programming","5000","1");
```

```
insert into Book values ("100002","Design and Analysis of  
Algorithm","Bijay","Programming","3500","2");
```

```
insert into Book values ("100003","Java Programming","Cijay","Programming","2500","3");
```

```
insert into Book values ("100004","Journey of Success","Jijay","Education","1000","4");
```

```
insert into Book values ("100005","Networking","Sanjay","Technology","2000","5");
```

```
insert into Book values ("100006","The Milestones","Vijay","Programming","1500","6");
```

```
delimiter $
```

```
create procedure Book()
```

```
begin
```

```
select * from Book;
```

```
end
```

```
$
```

```
delimiter ;
```

```
call Book();
```

```
delimiter $
```

```
create procedure Book1()
```

```
begin
```

```
select * from Book where category="Programming";
```

```
end
```

```
$
```

delimiter ;

call Book1();

delimiter \$

create procedure Book2(in jangovarchar(20))

begin

select * from Book where category=jango;

end

\$

delimiter ;

call Book2("Programming");

delimiter \$

create procedure Delete_book()

begin

delete from Book where pid=3;

end

\$

delimiter ;

callDelete_book;

Experiment No: 9

Aim: Develop a PL/SQL Program that explains the Trigger concept

Solution :

Trigger- After Insert

```
create table customer(id int , item varchar(20),cost int, quantity int);  
insert into customer values(101,"Pen",10,5),(102,"Soap",55,3);  
create table customer2(id int , item varchar(20),totalcostint);  
delimiter //  
create trigger trr_ins  
after insert on customer  
for each row  
begin  
insert into customer2 values(new.id,new.item,new.cost*new.quantity);  
end //  
delimiter ;  
select * from customer;  
select * from customer2;  
insert into customer values(103,"Book",40,5);  
select * from customer2;
```

Trigger- DuringDeletion

```
create table Student1(id int, name varchar(20), semint);  
insert into Student1 values(101,"Abin",2),(102,"Anand",2);  
delimiter //  
create trigger trr_del  
after delete on Student1  
for each row  
begin  
insert into Student2 values(old.id,old.name,old.sem,current_time());
```

```
end //
```

```
select * from Student1;
```

```
create table Student2(id int, name varchar(20), semint, logout varchar(20));
```

```
delete from Student1 where id=102;
```

```
delimiter ;
```

```
select * from Student2;
```

Experiment No: 10

Aim: Develop a PL/SQL Program that explains the Cursor concept

Solution:

Cursor :

```
delimiter $$
create procedure proc_emp()
begin
declare v_ename varchar(100);
declare v_salary int;
declare v_finished integer default 0;
declare c1 cursor for select name, salary from customers;
declare continue handler for NOT FOUND set v_finished=1;
open c1;
get_emp: LOOP
fetch c1 into v_ename, v_salary;
if v_finished=1 then
leave get_emp;
end if;
select concat(v_ename,concat('-',v_salary));
END LOOP get_emp;
close c1;
end $$
```


Experiment No :11

Aim : Develop a PL/SQL Program that explains the Function concept

Function :

```
CREATE DATABASE SPMCA;
```

```
USE SPMCA;
```

```
delimiter&
```

```
create function S_area(a int)
```

```
returnsint
```

```
deterministic
```

```
begin
```

```
return a*a;
```

```
end&
```

```
delimiter ;
```

```
selectS_Area(10);
```

```
delimiter&
```

```
create function R_Area(length int ,breadth int)
```

```
returns double
```

```
deterministic
```

```
begin
```

```
declare area int;
```

```
set area = length * breadth;
```

```
return area;
```

```
end&
```

```
delimiter ;
```

```
selectR_Area(10,2);
```

```
delimiter&
```

```
create function C_area(r int)
```

returns double

deterministic

begin

return 3.14*r*r;

end&

delimiter ;

selectC_area(10);

delimiter //

create procedure Area()

begin

selectR_Area(10,2);

selectS_Area(10);

selectC_area(10);

end //

delimiter ;

call Area();

Experiment No: 12

Aim: Design a NoSQL Database in MongoDB

Solution:

Create Database

```
use s2mca
```

Create Collection

```
db.createCollection("students")
```

Display the Collections

```
show collections
```

Insert documents

```
db.students.insertOne({"Name": "Daksh", "Age" : 2, "Location" : "Palakkad"})
```

Insert many documents

```
db.students.insertMany([  
  
  {  
  
    "Name": "Vimal",  
  
    "Age": 33,  
  
    "Location" : "Palakkad"  
  
  },  
  
  {  
  
    "Name": "Aneesh",  
  
    "Age": 39,  
  
    "Location" : "Trivandrum"  
  
  },  
  
  {  
  
    "Name": "Anil",
```

"Age": 35,

"Location" : "Thrissur"

}

)

Experiment No :13

Aim :CRUD Operations in NoSQL Databases (MongoDB)

Solution :

Create Operation :

Create Database

```
use s2mca
```

Create Collection

```
db.createCollection("students")
```

Insert documents

```
db.students.insertOne({"Name": "Daksh", "Age" : 2, "Location" : "Palakkad"})
```

Insert many documents

```
db.students.insertMany([
```

```
{
```

```
"Name": "Vimal",
```

```
"Age": 33,
```

```
"Location" : "Palakkad"
```

```
},
```

```
{
```

```
"Name": "Aneesh",
```

```
"Age": 39,
```

```
"Location" : "Trivandrum"
```

```
},
```

```
{
```

```
"Name": "Anil",
```

```
"Age": 35,
```

```
"Location" : "Thrissur"
```

```
}
```

```
])
```

ReadOperation :

Selecting the document/Reading

```
db.students.find()
```

Selection base upon filter query

```
db.students.find(
```

```
{
```

```
  Name : "Daksh"
```

```
}
```

```
)
```

UpdateOperation :

Updating the documents

```
db.students.update({Age : 39}, {$set: {Age: 23}})
```

DeleteOperation :

Deleting the documents

```
db.students.deleteOne({Name : "Anil"})
```

Experiment No: 14

Aim:Usage of aggregate functions in NoSQL

Solution :

Create a collection named as staff

```
db.createCollection("staff")
```

Insert documents into collection named as staff

```
db.staff.insertMany([
{
  _id:1,
  firstName: "Ajay",
  lastName: "Sankar",
  gender:'male',
  email: "ajaysankar@gmail.com",
  salary: 5000,
  department: {
    "name":"HR"
  }
},
{
  _id:2,
  firstName: "Sachin",
  lastName: "T",
  gender:'male',
  email: "sachinay@gmail.com",
  salary: 8000,
  department: {
    "name":"Finance"
  }
}]
```

```
},  
  
{  
  _id:3,  
  firstName: "David",  
  lastName: "John",  
  gender:'male',  
  email: "davidjohn@gmail.com",  
  salary: 7500,  
  department: {  
    "name":"Marketing"  
  }  
},  
  
{  
  _id:4,  
  firstName: "Indu",  
  lastName: "Lekha",  
  gender:'female',  
  email: "indulekha@gmail.com",  
  salary: 5000,  
  department: {  
    "name":"HR"  
  }  
},  
  
{  
  _id:5,  
  firstName: "Jagan",  
  lastName: "Mohan",  
  gender:'male',  
  email: "jaganmohan@gmail.com",  
  salary: 4500,
```



```
department: {
  "name": "Finance"
}
},
{
  _id: 6,
  firstName: "Derik",
  lastName: "Abraham",
  gender: 'male',
  email: "derikabraham@gmail.com",
  salary: 7000,
  department: {
    "name": "Marketing"
  }
}
])
```

\$group Stage

```
db.staff.aggregate([
  { $group: { _id: '$department.name' } }
])
```

\$match Stage

```
db.staff.aggregate([
  { $match: { gender: 'female' } }
])
```

\$sum & \$group (Finding the total no. of employees in each department)

```
db.staff.aggregate([
```

```
{ $group:{ _id:$department.name', totalEmployees: { $sum:1 } }  
})
```

\$match, \$group & \$sum

```
db.staff.aggregate([  
  { $match:{ gender:'male'}},  
  { $group:{ _id:$department.name', totalEmployees: { $sum:1 } }  
})
```

Finding the total salary of employees in each department

```
db.staff.aggregate([  
  { $match:{ gender:'male'}},  
  { $group:{ _id:{ deptName:$department.name'}, totalSalaries: { $sum:$salary' } }  
})
```

\$sort (sorting in ascending order according to firstName)

```
db.staff.aggregate([  
  { $match:{ gender:'male'}},  
  { $sort:{ firstName:1}}  
])
```

\$sort (sorting in descending order according to firstName)

```
db.staff.aggregate([  
  { $match:{ gender:'male'}},  
  { $sort:{ firstName:-1}}  
])
```

Experiment No: 15

Aim:Usage of regular expression in NoSQL

Solution :

Create collection named as emp

```
db.createCollection("emp")
```

Insert documents into collection emp

```
db.emp.insertMany([
{
Name: "Ajay",
position: "Backend developer"
},
{
Name: "Babu",
position: "Frontend developer"
},
{
Name: "Christy",
position: "HR Manager"
}
])
```

\$regex

```
db.emp.find({position : {$regex : "developer"}})
```

Displaying the documents of those employees whose name starts with 'B' letter

```
db.emp.find({Name : {$regex : "^B"}})
```

Displaying the documents of those employees whose names end with the 'y' letter

```
db.emp.find({Name : {$regex : "y$"}})
```

Displaying the documents of those employees whose position contain the “end” string

```
db.emp.find({position : /end/})
```

Displaying the documents of those employees whose position contain the “Front” string

```
db.emp.find({position : /Front/})
```

i: To match both lower case and upper case pattern in the string

```
db.emp.find({position : {$regex : "front", $options:"i"}})
```

Experiment No: 16

Aim:Creating users and setting roles in NoSQL

Solution :

Creating a user administrator

```
db.createUser(  
  { user: "daksh",  
    pwd: "daksh123",  
    roles:[{role: "userAdminAnyDatabase" , db:"admin"}}})
```

Create User for Single Database

```
db.createUser(  
  {  
    user: "kapila",  
    pwd: "mca",  
    roles:[{role: "userAdmin" , db:"s2mca"}}})
```

Managing users

```
db.createUser(  
  {  
    user: "sanjay",  
    pwd: "123",  
    roles:[  
      {  
        role: "read" , db:"abc"},  
      {  
        role: "readWrite" , db:"s2mca"}  
    ]  
  })
```

List out the existing users

```
db. getUsers()
```

Drop a user

```
db.dropUser("sanjay")
```

Experiment No: 17

Aim:NoSQL Administration security & backup

Solution:

Verify the authentication

```
db.auth("daksh", "daksh123")
```

Backup & Restore of database (First of all install MongoDB Tools and add its path of bin folder to environment variable)

Backing up the specified collection(Type the below command in normal cmd)

```
mongodump --db s2mca --collection students --out c:\mbst
```

Delete the collection named as students from the database s2mca from the MongoDB shell

```
use s2mca
```

```
db.students.drop()
```

Restore the collection students from the backup you have already created(Type the below command in normal cmd)

```
mongorestore --db s2mca --collection students c:\mbst\s2mca\students.bson
```

Experiment No :18

Aim:NoSQL: MongoDB& Front-end: Java Connectivity

Solution:

First of all we need to install Eclipse then you have to download the mongo-java-driver-3.12.14.jar driver for connecting java code with MongoDB

Code inside MongoMain.java

```
import com.mongodb.*;

public class MongoMain {

    public static void main(String[] args) {

        try {

            MongoClient mongo = new MongoClient("localhost",27017);

            DB db1 = mongo.getDB("s2mca");

            DBCollection coll = db1.getCollection("emp");

            BasicDBObject d1 = new
BasicDBObject("Name","Ajay").append("position","Backend developer");

            BasicDBObject d2 = new
BasicDBObject("Name","Babu").append("position","Frontend developer");

            coll.insert(d1);

            coll.insert(d2);

            DBCursor cur = coll.find();

            while(cur.hasNext())
            {
                System.out.print(cur.next());
            }

        }
        catch(Exception ex) {

            ex.printStackTrace();

        }

    }

}
```


}

}

Output :

The screenshot shows the MongoDB Compass interface for a local instance at localhost:27017. The left sidebar displays the database structure, with the 'Employee' database and 'emp' collection highlighted. The main panel shows the 'Employee.emp' collection with two documents. The first document is for 'Ajith' (Backend developer) and the second is for 'Bibin' (Frontend developer). Both documents are highlighted with red boxes.

Employee.emp

2 DOCUMENTS 1 INDEXES

Filter Type a query: { field: 'value' }

ADD DATA EXPORT DATA

1 - 2 of 2

<pre>{ "_id": ObjectId("64b76c312bf39c1bbca4423b"), "Name": "Ajith", "position": "Backend developer" }</pre>
<pre>{ "_id": ObjectId("64b76c322bf39c1bbca4423c"), "Name": "Bibin", "position": "Frontend developer" }</pre>